

JudgeMesh 分布式在线判题系统



分布式软件原理与技术课程期末项目汇报

汇报人：肖天

汇报日期：2026 年 06 月 03 日

目录



01 软件功能

明确项目定位与核心技术栈，解析微服务架构的拆分逻辑，深度梳理系统支撑的核心业务场景与价值边界。



02 问题分析

剖析高并发、高可用、高性能场景下的核心矛盾，梳理当前技术仓库中已暴露的架构瓶颈与业务边界问题。



03 技术方案

构建端到端的总体可靠链路，实施削峰重试、分布式调度选主机制，并通过统一网关鉴权筑牢系统安全防线。



04 测试评估

确立统一测试口径，开展多维度性能压测与故障恢复演练，基于实测数据形成系统稳定性的综合评估结论。

01 软件功能 | 项目定位与技术栈

项目定位：面向课程大作业的分布式在线判题系统

系统完整覆盖题目管理、提交编译、比赛组织、实时榜单、任务调度、沙箱判题、全链路可观测与故障容灾演示等核心场景，为分布式系统课程实践提供高可用、可扩展的一体化解决方案。

后端核心栈

基于 JDK 17 构建，采用 Spring Boot 3.5.0 与 Spring Cloud Alibaba 2025 全家桶，结合 MyBatis-Plus 实现高效的数据访问层，打造云原生微服务架构。

存储与中间件

MySQL 8 存储核心数据，Redis 7 做缓存与会话管理；RabbitMQ 削峰填谷，MinIO 存储测试用例；集成 Nacos 服务治理、Sentinel 流量控制与 Seata 分布式事务。

判题与交互层

Go 1.22 编写 judge-worker，isolate 沙箱实现安全隔离；前端采用 React 19 + TypeScript + Vite 8 构建现代化交互，Nginx 负责反向代理与静态资源分发。

运维与可观测

基于 Kubernetes 1.28 容器化部署，KEDA 实现弹性伸缩；集成 Prometheus + Grafana 监控，SkyWalking 链路追踪，Loki 日志聚合，GitHub Actions 自动化 CI/CD。

01 软件功能 | 微服务模块拆分

gateway · 统一网关入口

作为系统统一请求入口，负责 JWT 身份校验、Header 信息透传、接口限流熔断及路由转发，保障后端服务安全与稳定。

submit-service · 提交调度服务

处理提交记录创建、状态流转管理，支持比赛报名、全站/比赛排行榜计算，并通过 WebSocket 实时推送提交状态。

user-service · 用户核心服务

专注用户全生命周期管理，涵盖注册登录、JWT 凭证签发、用户资料维护及账户余额扣减等核心业务逻辑。

judge-dispatcher · 判题任务分发

作为判题任务中枢，负责消费提交任务，利用 etcd 实现选主与 Worker 节点负载均衡，处理失败重试与死信队列(DLQ)。

problem-service · 题目资源服务

统筹题目全量数据，提供题目列表与详情展示、标签分类管理，并对接 MinIO 实现测试用例 manifest 对象存储管理。

judge-worker · 判题执行引擎

基于 Go 语言实现，支持 C/C++、Java、Python 等多语言代码编译运行，完成输出比对与判题结果回写，确保判题准确性。

01 软件功能 | 五类业务一：用户认证与权限



功能设计：全流程身份管理

支持用户注册、登录核心流程，集成JWT签发与校验机制；实现当前用户信息快速查询，并严格区分 STUDENT、SETTER、ADMIN 三类核心角色，满足不同业务场景的权限管控需求。



网关处理：统一鉴权与透传

Gateway层统一拦截并验证Bearer Token的有效性，通过HTTP Header向下游微服务透传 X-User-Id、X-User-Name、X-User-Roles 核心信息，简化下游服务的身份处理逻辑。



存储方案：安全加密存储体系

用户基础信息与角色映射关系持久化存储于 users_db；敏感密码采用 BCrypt 哈希算法加密，Token 则选用 HMAC-SHA256 算法进行自签生成，保障身份凭证的安全性。



业务价值：降本增效与架构统一

构建统一的认证入口，收敛鉴权逻辑至网关层，避免各业务服务重复开发身份校验模块，有效降低系统冗余代码与维护成本，提升整体架构的一致性与安全性。

01 软件功能 | 五类业务二：题目与测试用例



功能设计

系统完整支持题目列表展示、题目详情预览、创建与编辑操作，同时提供灵活的标签管理功能，并实现测试用例文件上传及manifest文件的便捷查询，覆盖全生命周期管理需求。



存储方案

采用分层存储策略：结构化的题目元数据存储于MySQL的`problems\_db`中，便于高效检索；非结构化的大体积测试用例文件则存放于MinIO对象存储的`testcases/`桶中，兼顾成本与读写性能。



读优化策略

针对题目“读多写少”的典型特征，将高频访问的题目详情数据缓存至Redis中；热门题目列表配置定时任务刷新机制，大幅减少数据库直连请求，提升系统整体响应速度。



判题逻辑解耦

判题Worker进程仅与测试用例的manifest描述文件及MinIO中的用例对象交互，完全不直接耦合后台管理系统的业务逻辑，保障了判题服务的稳定性与独立性，降低系统风险。

01 软件功能 | 五类业务三：提交、判题与比赛



功能设计：异步判题与状态流转

学生提交代码后，系统即时写入**PENDING**状态，任务异步进入判题链路，最终完成 AC、WA、TLE、MLE、RE 等多维度的状态闭环流转。



比赛能力：全流程赛事闭环支持

完整支持比赛创建、用户报名、比赛题集配置等基础能力，同时内置封榜逻辑与比赛期间的提交校验机制，保障赛事的公平性与规范性。



存储方案：分层存储与异步解耦

提交记录与比赛主数据持久化落地**submits_db**，判题任务则通过 RabbitMQ 的 submit.queue 队列异步分发，实现业务层与执行层的解耦。



防重策略：高频提交限流管控

针对同一用户同一题目，设置**1秒内禁止重复提交**的限制策略，有效规避用户短时误触操作，同时防止恶意刷接口行为，保护系统稳定性。

01 软件功能 | 五类业务四、五：排行榜与分布式判题

排行榜功能

支持全站榜、比赛榜多维度展示，数据实时刷新。全站榜单采用“AC 数优先，总耗时次之”的核心排序规则，兼顾解题数量与代码效率，呈现精准排名。

高性能存储

利用 Redis ZSet 维护核心榜单，Hash 与 Set 结构存储详情与去重数据；通过 WebSocket 协议实现数据的实时推送，确保前端榜单毫秒级更新，提升用户体验。

分布式调度

Dispatcher 作为协调者选主，Leader 节点消费任务队列并分发给 Worker 池并行处理；任务完成后 Worker 回调 submit-service 接口，统一完成结果落库，保障流程闭环。

全链路观测

基于 Grafana 监控 QPS、判题延迟、Worker 水位及队列堆积情况；结合 SkyWalking 实现全链路追踪，精准定位系统瓶颈，保障平台高可用性与稳定性。

通过分层架构设计，实现了排行榜的实时性与分布式判题的高吞吐、高可用，配合完善的可观测体系，构建了稳定、高效、透明的核心业务闭环。

02 问题分析 | 三高场景下的核心矛盾



流量与计算的矛盾

提交流量呈突发高峰特性，而判题执行属于CPU/IO密集型的慢操作，两者直接耦合会导致系统过载。必须通过异步化手段，将“提交入口”和“执行计算”彻底解耦。



高可用与一致性

Dispatcher组件需要多副本部署以保证高可用，但多副本并发消费时极易出现同一提交被重复派发的问题，这对分布式系统的一致性保障提出了严格要求。



资源与目标的矛盾

课堂项目的云资源配额有限，架构设计既要完整展示高可用、弹性伸缩与熔断降级等核心能力，又不能引入过多未经验证的中间件集群，增加维护成本。

P99 < 200ms

提交同步阶段响应耗时

P99 < 8s

端到端完整判题链路耗时

> 100 QPS

系统常态稳定吞吐量

> 500 QPS

比赛场景峰值并发承载

03 技术方案 | 总体可靠链路

入口层：流量接入与安全管控

请求链路：Browser → Ingress-nginx → Gateway。前置完成 JWT 身份校验、公开路由直接放行，同时对提交接口实施精细化限流策略，保障系统入口稳定性。

业务层：异步解耦与快速响应

submit-service 优先将提交记录写入 submission 表并置为 PENDING 状态，随后投递任务至 RabbitMQ 实现异步解耦，确保用户请求毫秒级响应，核心业务不阻塞。

协调层：分布式选主与任务分发

judge-dispatcher 基于 etcd Lease + Election 机制实现分布式选主，仅 Leader 节点负责从 MQ 消费任务并分发给执行节点，避免重复消费，保障分发一致性。

执行层：隔离运行与全链路更新

Go Worker 拉取用例在 isolate 沙箱中安全运行，执行完成后回调 submit-service，由其统一更新 MySQL 结果、Redis 缓存，并通过 WebSocket 实时推送判题状态至前端。

核心配置：judgemesh.etcd.leader-key = /judgemesh/dispatcher/leader | lease-ttl-seconds = 10s (确保节点故障时快速重新选主)

03 技术方案 | 提交削峰、重试与恢复

01 削峰核心：多级缓冲队列体系

以 RabbitMQ 为提交入口缓冲层，设计主队列 `submit.queue`、重试队列 `submit.retry.queue` 与死信队列 `submit.dlq` 三级结构，有效缓冲瞬时流量冲击，实现任务的有序分发与沉淀。

02 重试策略：有限重试与兜底机制

`dispatcher` 任务派发失败时，系统自动执行最多 3 次重试；若仍未成功，则将状态回写为 SE (Submit Error)，并将任务投递至死信队列，避免无限循环占用资源。

03 恢复策略：主动扫描与超时自愈

`submit-service` 每 5 秒扫描一次状态为 JUDGING 的超时记录，若任务 30 秒未完成则触发自动恢复流程，最多执行 2 次自动重投，最大程度保障任务的最终完成率。

04 降级策略：旁路调用保障链路

当 MQ 集群发布失败或不可用时，`submit-service` 会自动切换策略，直接调用 `dispatcher` 的紧急派发端点，跳过消息队列缓冲，确保课堂演示等核心业务链路不中断。

```
judgemesh:mq: { submit-queue: submit.queue, retry-queue: submit.retry.queue, dead-queue: submit.dlq, retry-delay-ms: 5000 }
```

03 技术方案 | 调度选主、Worker 池与自动扩容



01. Worker 智能选择策略

系统优先过滤处于30秒黑名单的故障节点，再从健康节点中挑选当前 inflight 任务数最少的 Worker 分配请求，确保负载绝对均衡，避免单点过载。



02. 极简横向并发模型

严格遵循 **1 Pod = 1 并发** 的设计原则，不在单 Pod 内堆高并发。将系统的弹性伸缩能力完全解耦并下沉到 Kubernetes Pod 横向扩容层，保障运行稳定性。



03. 高可用集群部署架构

采用 **1 App 节点 + 3 Judge 节点** 的基础高可用配置；Judge-dispatcher 组件固定3副本部署以确保调度高可用；Worker 层则通过 KEDA 组件，基于消息队列长度实现全自动化的弹性扩缩容。



04. 精细化自动扩缩容参数

minReplicaCount:**3** | maxReplicaCount:**8**
cooldownPeriod:**180s** | 触发阈值 (Queue Length):**4**

03 技术方案 | 网关鉴权、限流与前端部署



鉴权方案：分级访问控制

公开接口仅保留登录、注册、题目浏览基础功能；核心业务接口统一校验 Bearer Token，确保身份合法性与数据安全。



工程价值：架构清晰易维护

前后端部署边界清晰，课堂答辩时可直观展示“页面静态托管 + 动态 API/WS 分流”的完整架构，既保证了访问性能，也便于后续扩展与运维。



限流方案：多级兜底保护

提交接口采用“用户ID优先、IP兜底”策略，单主体限频30次/分；Redis集群故障时自动降级为本地Caffeine计数器，保障服务可用性。



前端部署：动静分离架构

React 构建静态资源由 Nginx 直接托管，通过反向代理将 /api/ 转发至网关，/ws 路径升级为 WebSocket 协议，实现高效通信。

Nginx 核心配置片段

```
location /api/ { proxy_pass
http://gateway.judgemesh.svc.cluster.local; }
location /ws { proxy_pass
http://gateway.judgemesh.svc.cluster.local; }
location / { try_files $uri $uri/ /index.html; }
# 静态资源直接返回，动态请求反向代理，实现完整的BFF分层
```

04 测试评估 | 测试口径与环境



测试口径：对齐核心策略

严格遵循仓库「10-测试策略」规范，核心覆盖提交接口纯QPS压测、端到端判题全链路验证、高频排行榜查询、题目详情读取及故障恢复容灾演练五大核心场景。



测试工具：分层工具体系

采用 wrk 实施高并发接口压测，通过 demo-distributed-load.mjs 实现分布式判题与 Worker 负载分布；利用 JMeter 生成可视化性能报表，实现测试过程可观测、结果可追溯。



典型环境：4节点高可用拓扑

部署为2 App + 2 Judge节点的答辩版拓扑，配置 judge-dispatcher 3副本、judge-worker 3~8弹性副本，单实例 JUDGE_MAX_CONCURRENCY=1，保障判题资源隔离。



目标指标：P99性能达标验证

提交同步 / 端到端判题

P99 < 200ms / < 8s

榜单查询 / 题目详情

P99 < 50ms / < 100ms

04 测试评估 | wrk 提交接口纯 QPS 压测



测试对象与核心逻辑

目标接口：POST /api/submit。仅验证同步阶段，即完成“提交记录落库 + 任务入队 RabbitMQ”的完整链路处理。



压测环境与执行配置

执行命令：wrk -t4 -c50 -d300s -s scripts/loadtest-submit.lua。4线程50并发，持续压测5分钟，模拟真实高频提交场景。



平均响应时间 (RT)

42.37ms



长尾响应表现 (P90/P99)

78/156ms



吞吐能力与错误率

124.68req/s (0%错误)



测试结论：提交入口已与慢判题流程解耦，峰值流量实现“先落库再异步入队”。

核心指标 P99 响应时间控制在 156ms，完全满足“提交同步阶段 P99 < 200ms”的性能目标，系统在高频写入场景下表现稳定可靠。

04 测试评估 | 分布式端到端判题压测

全链路测试闭环

覆盖用户登录、题目提交、MQ消息派发、Worker节点判题、结果回写DB及前端状态轮询的完整业务流程。

高并发场景压测配置

执行脚本：node scripts/demo-distributed-load.mjs，配置总计60用户，并发数30，模拟真实高频提交场景。

平均响应时间 (RT)

1864_{ms}

P90 响应时间

2987_{ms}

P95 响应时间

3721_{ms}

P99 响应时间

6418_{ms}

最大响应耗时

7346_{ms}

判题结果分布：状态全AC，零异常

statusCounts={"AC":60} —— 系统稳定性验证通过，无判题错误或超时丢失。

Worker节点负载：绝对均衡，压力均分

workerCounts={"worker-1":20,"worker-2":20,"worker-3":20} —— 任务调度算法有效性验证。

04 测试评估 | 故障恢复与高可用验证



模拟单节点故障场景

手工强制停止 worker-2 节点，验证调度器的节点黑名单机制，以及剩余节点（worker-1/3）自动接管任务的容错能力。



分布式高并发压测配置

执行脚本: node scripts/demo-distributed-load.mjs
参数配置: --total 30 (总请求) --concurrency 15 (并发) --users 30 (虚拟用户)

平均响应时间 (RT)

2149_{ms}

P90 响应时间

3364_{ms}

P95 响应时间

4012_{ms}

P99 长尾响应

6895_{ms}

任务状态分布:
全量成功 (AC:30)

存活节点负载:
worker-1/3 均分 (15/15)



结论：单 Worker 宕机未引发流程中断，系统实现了请求自动重分发，具备可靠的基础高可用能力。

04 测试评估 | JMeter 排行榜与题目详情压测

场景一：排行榜查询GET /api/rank/global

平均 RT
11.82ms

P99 延迟
43ms

吞吐量:**1386.24** req/s 错误率:**0%**

场景二：题目详情GET /api/problem/{id}

平均 RT
26.41ms

P99 延迟
87ms

吞吐量:**1642.57** req/s 错误率:**0%**



缓存效果显著：Redis 命中率 95%+
读请求基本不穿透 MySQL，极大减轻了数据库的负载压力，确保了核心读接口的高并发响应能力。



压测结论：性能达标，系统稳定
读多写少业务场景已通过多级缓存架构，完美达成低延迟、高吞吐的技术指标，错误率为零，系统表现优异。

04 测试评估 | 综合结论

功能闭环验证

成功完成smoke-test.sh自动化冒烟测试，Python/C++ A+B样例、分布式任务派发及单节点故障自动恢复流程均可完整演示，核心业务链路无断点。

关键性能达标

提交同步阶段延迟稳定在百毫秒级；端到端判题P99控制在**8s**以内；Redis缓存层让榜单与题目详情查询延迟显著降低，用户体验流畅。

架构方案成效

RabbitMQ实现了流量削峰填谷，etcd选主机制保障高可用，worker池支持横向弹性扩容，Redis排行榜与多级缓存方案有效支撑了高并发读请求。

系统稳定性优异

全量答辩版测试覆盖核心场景，结果错误率均为**0%**，验证了系统在课程目标负载下具备极高的稳定性与服务可用性，边界条件处理完善。

未来优化方向

持续补充长时间高负载容量压测，完善etcd节点故障与网络分区的恢复演练；引入KEDA实现基于消息队列长度的自动扩缩容，并补充更多JMeter多维度性能报表数据。

THE END

A decorative graphic featuring a horizontal green line and several green bokeh circles of varying sizes, centered around the text.

感谢聆听